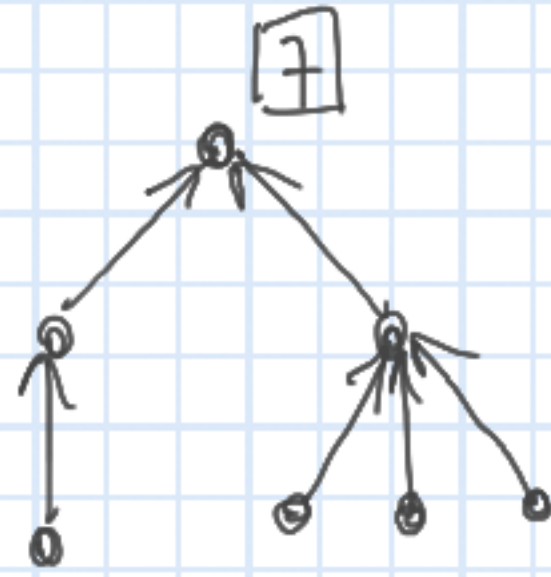


November 26, 2021

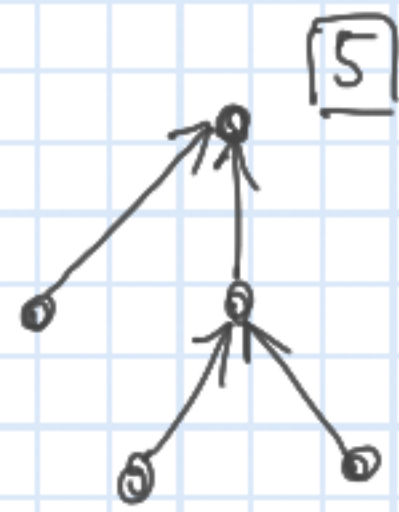
Union - find implementation

① "Visually"

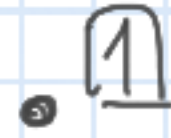
Each u_i can be represented by a bottom up directed rooted tree with a tag at the root containing the number of vertices of the tree.



u_1



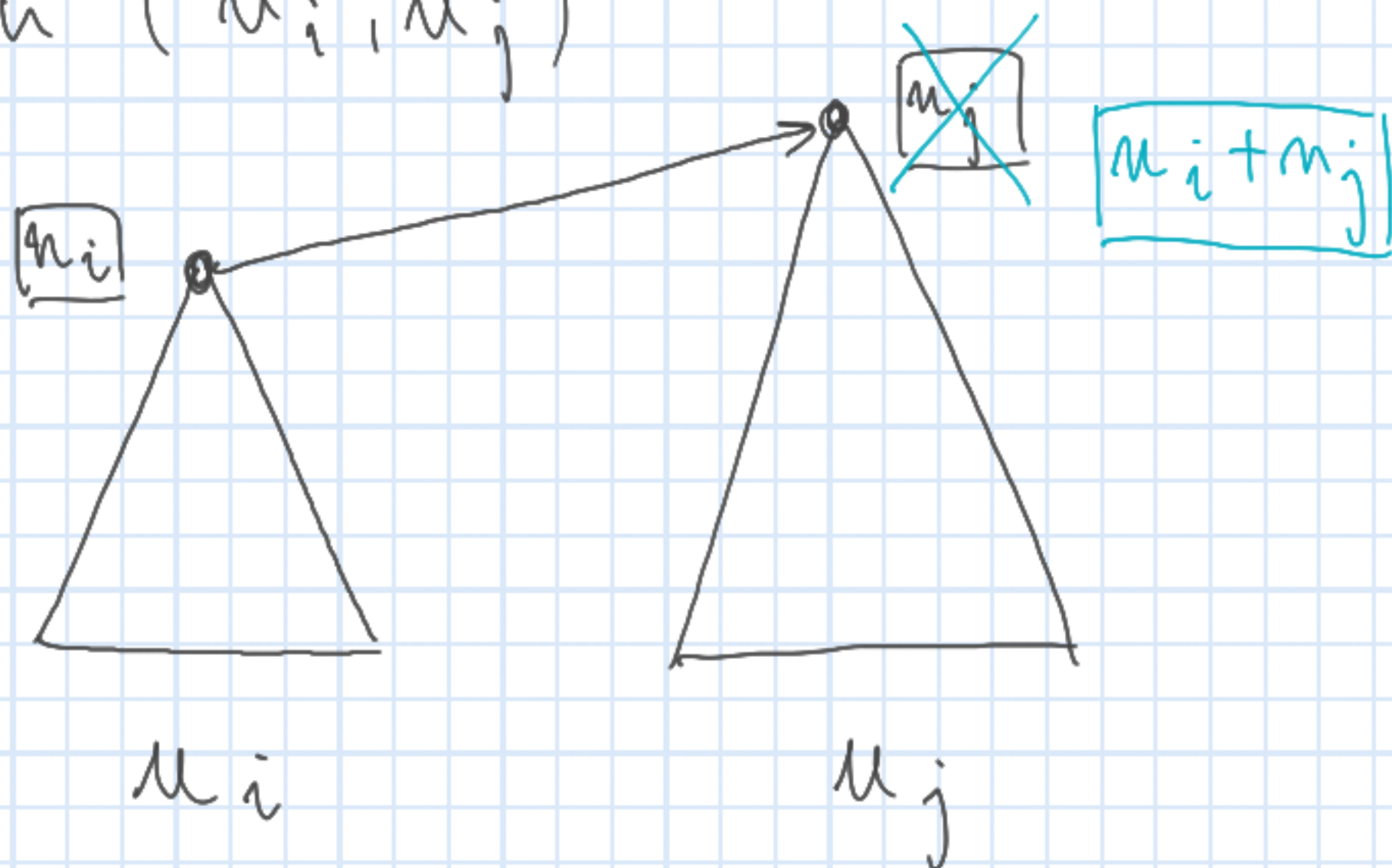
u_2



u_3

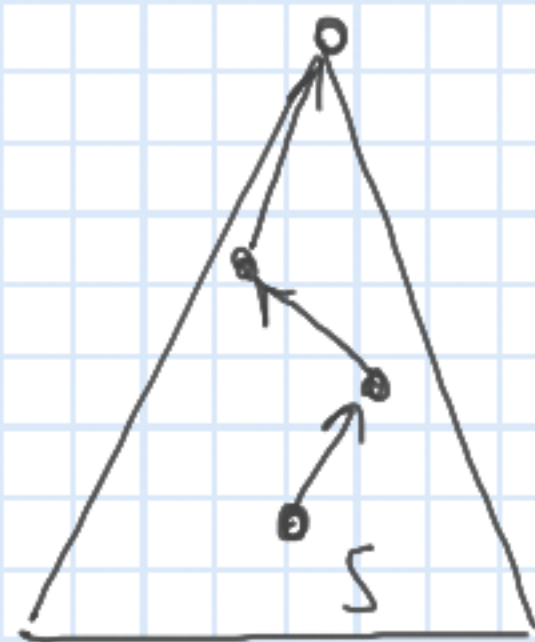
In the typical applications (as Kruskal alg.)
we start with the partition consisting of
singletons (one-element subsets) and we
execute a sequence of finds and unions

Union (u_i, u_j)



$$u_i \leq u_j$$

Find (s)



follow the parent pointers
to reach the root; the root
identifies the set containing
 s

Cost

Union : $O(1)$ time

Find : $O(\log n)$ time

② "Technically"

We use an array indexed by the objects (vertices). The elements of the array are pairs consisting a number (of element of the subtree) and an array index (pointing to the parent of the vertex)



corresponding array

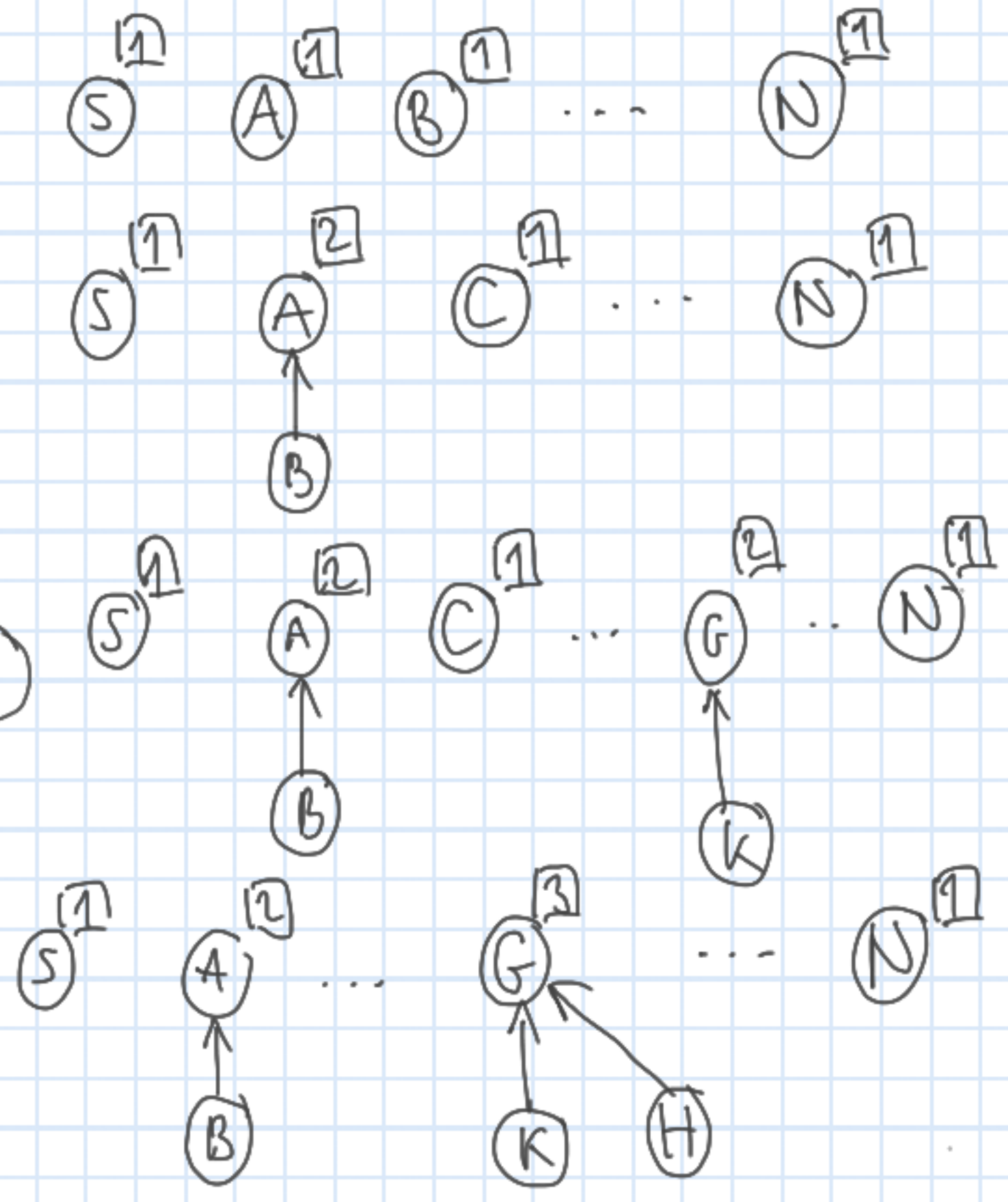
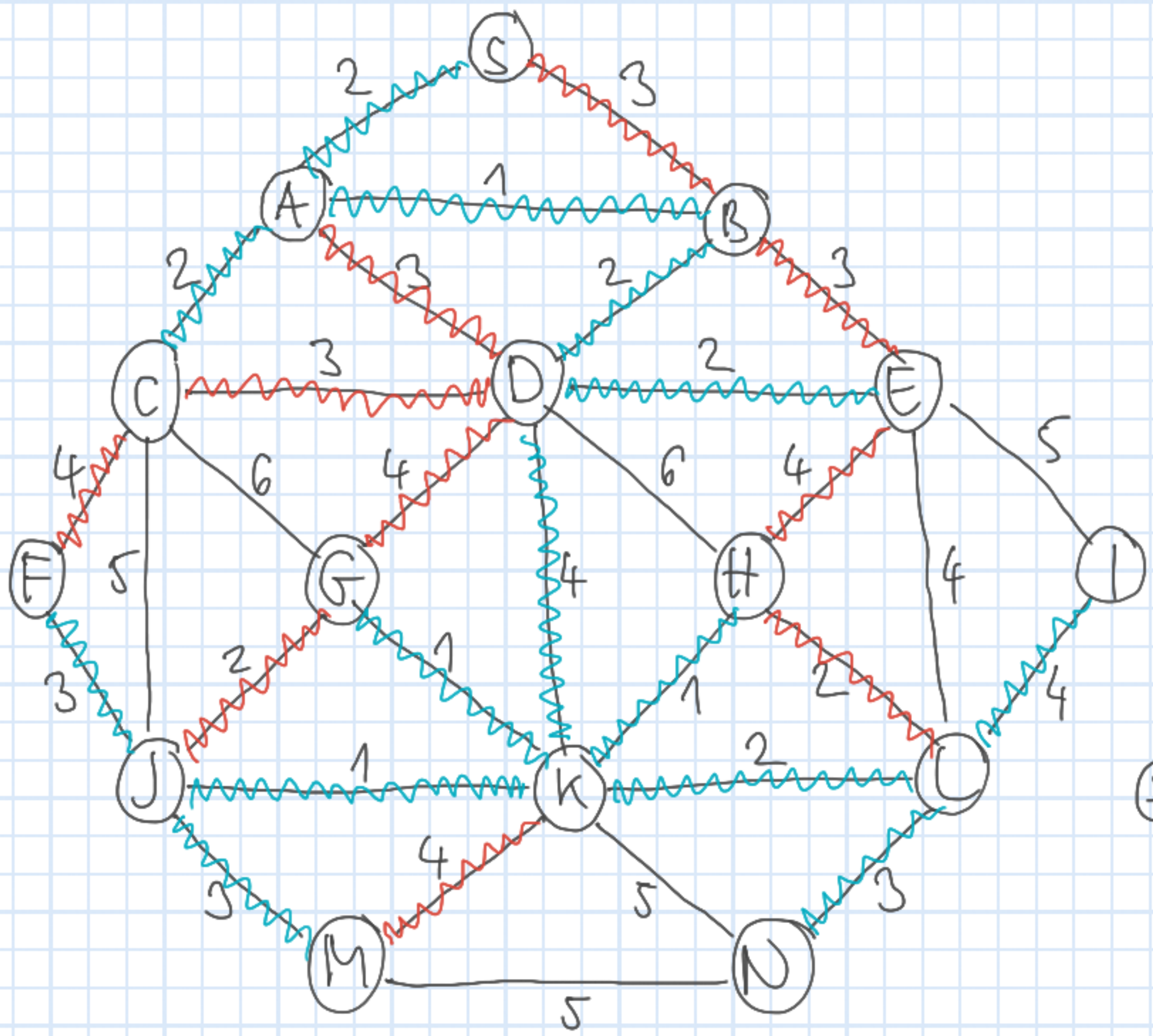
size

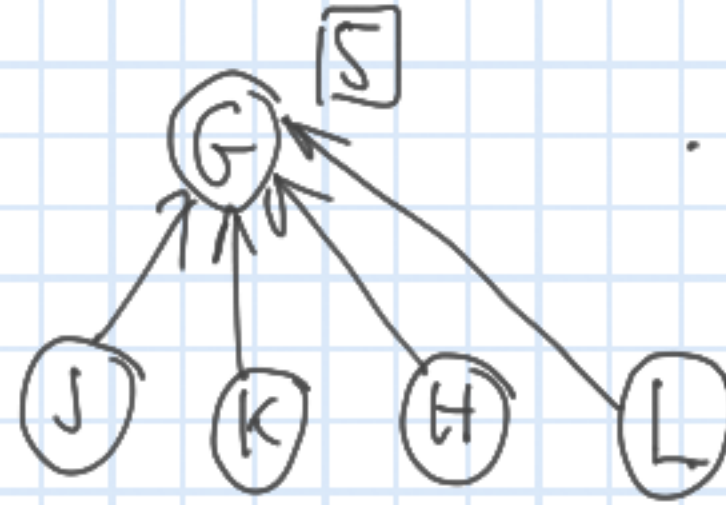
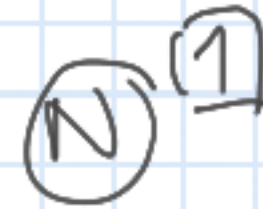
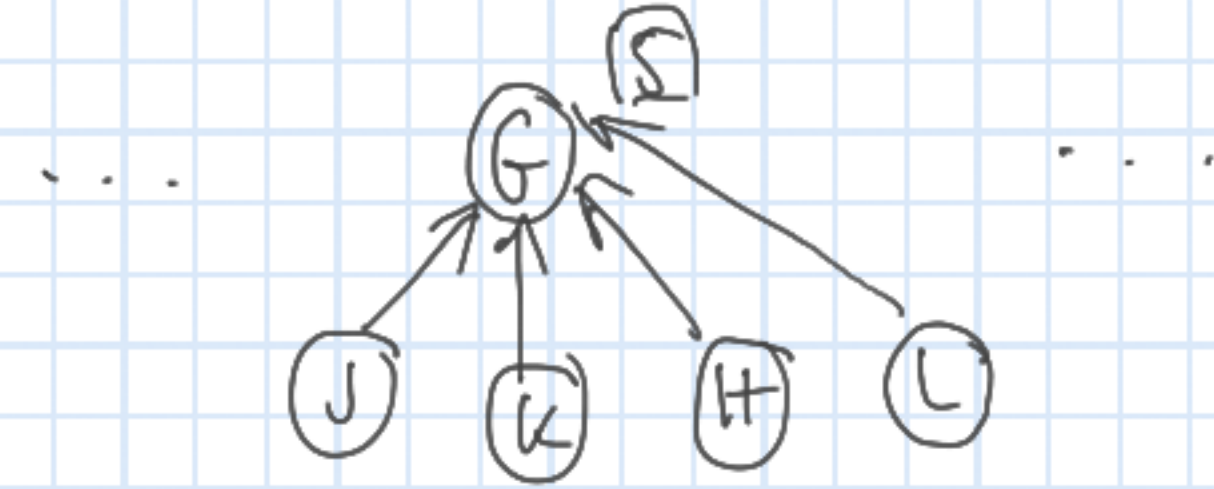
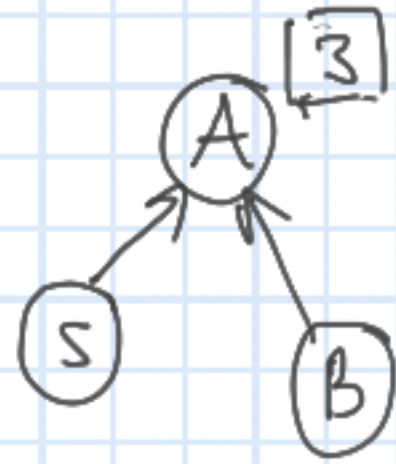
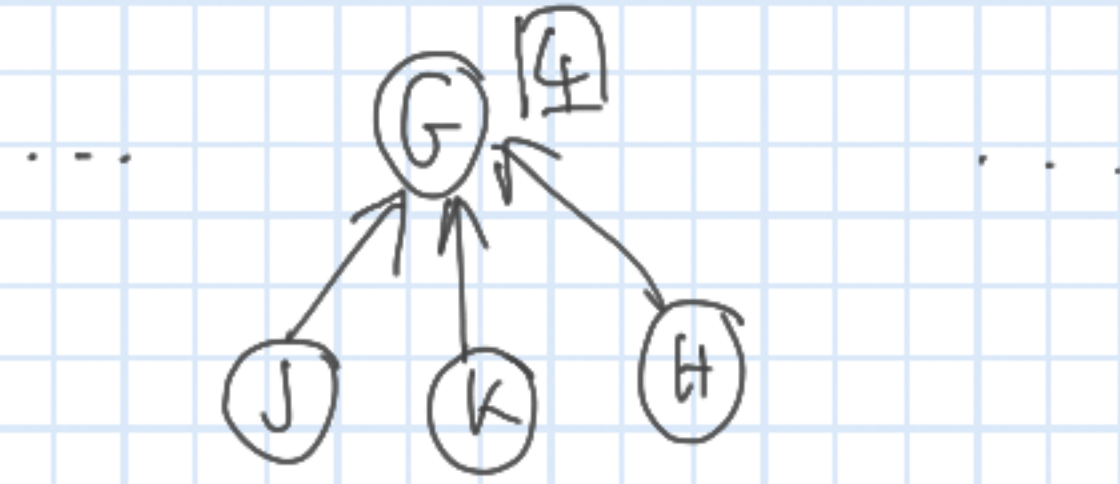
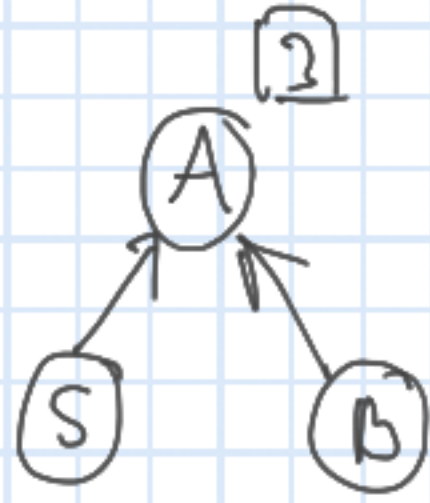
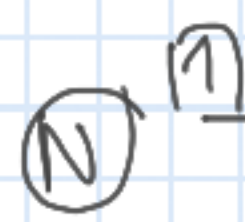
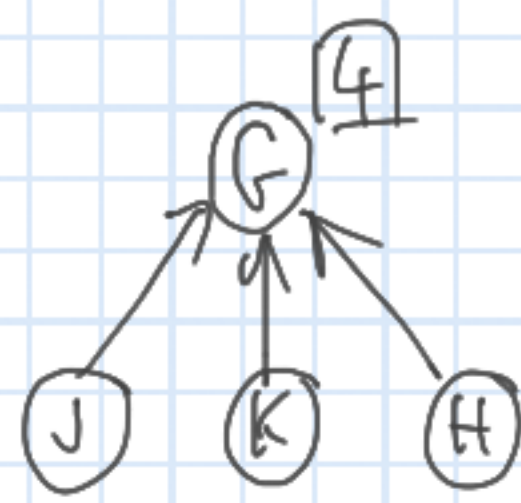
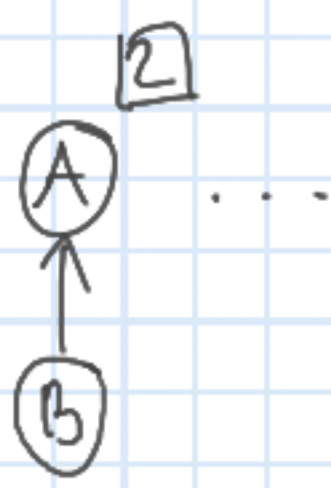
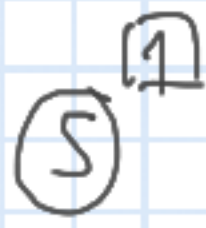
size is dummy if the parent is not nil

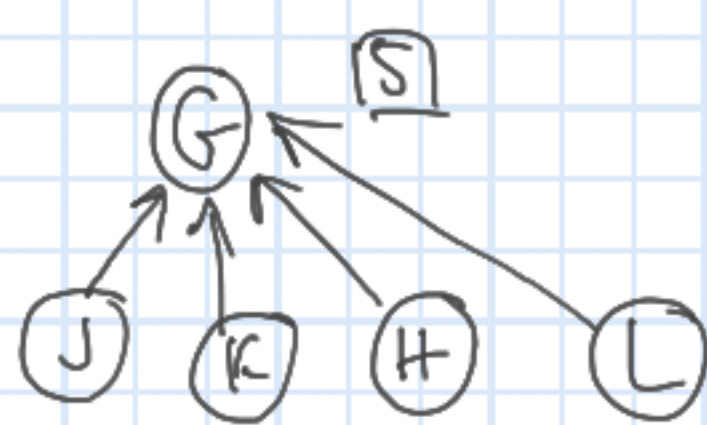
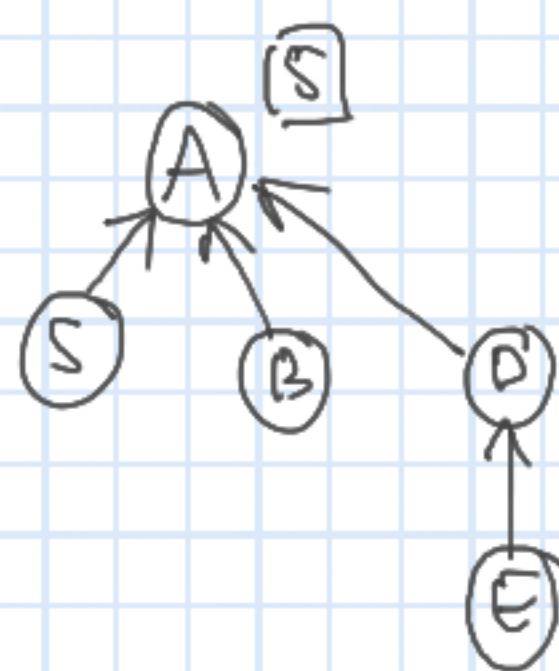
Initially :

[illegible]

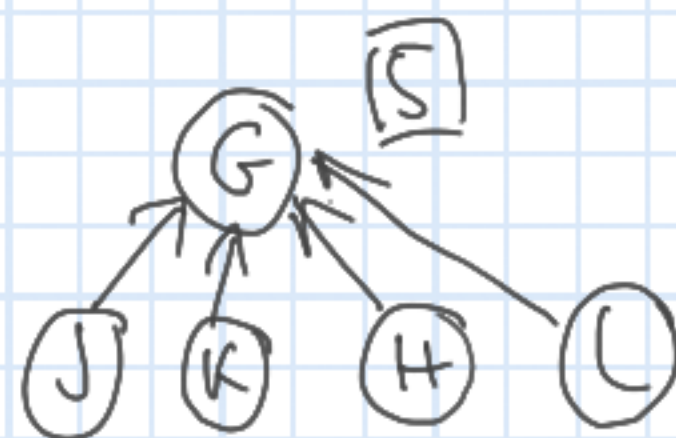
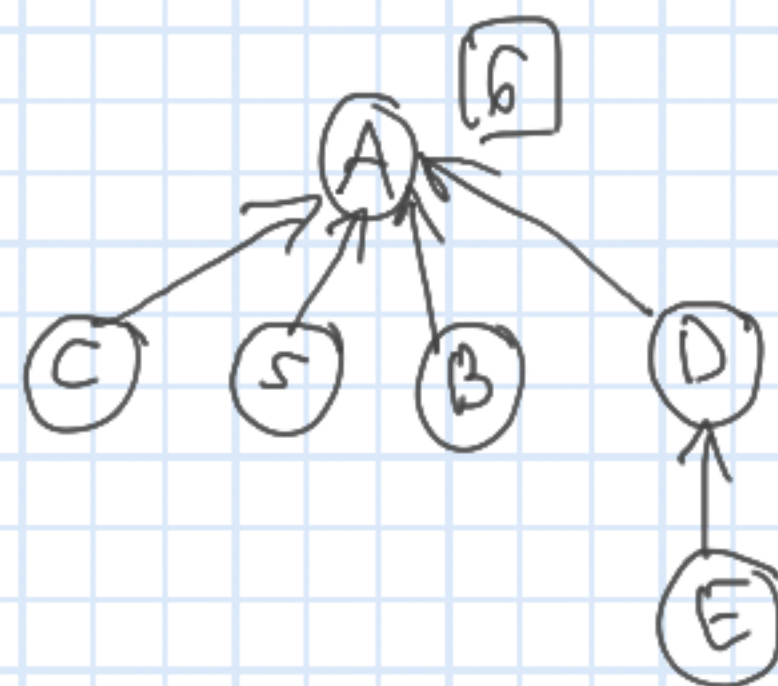
Union-Find



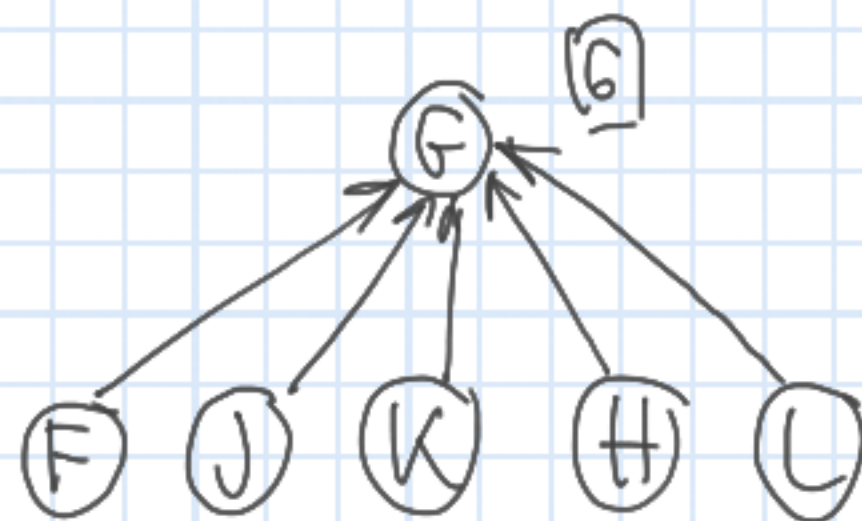
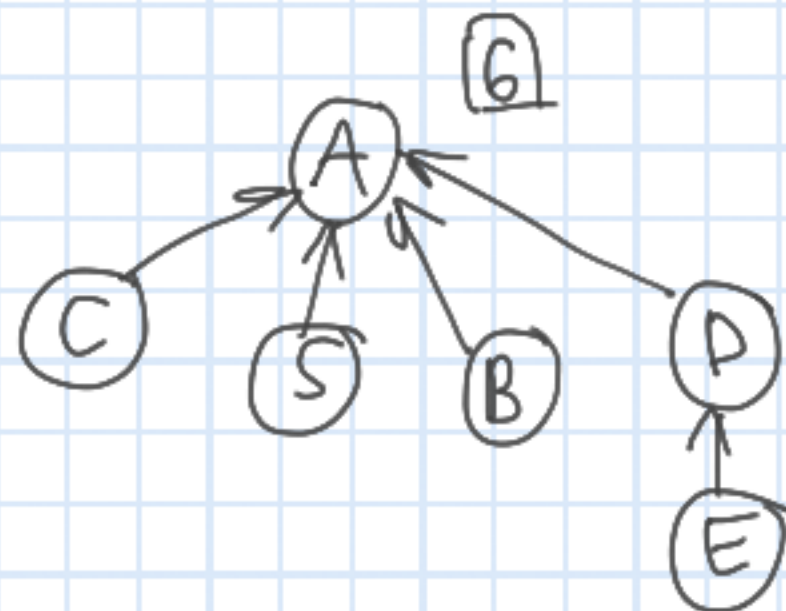




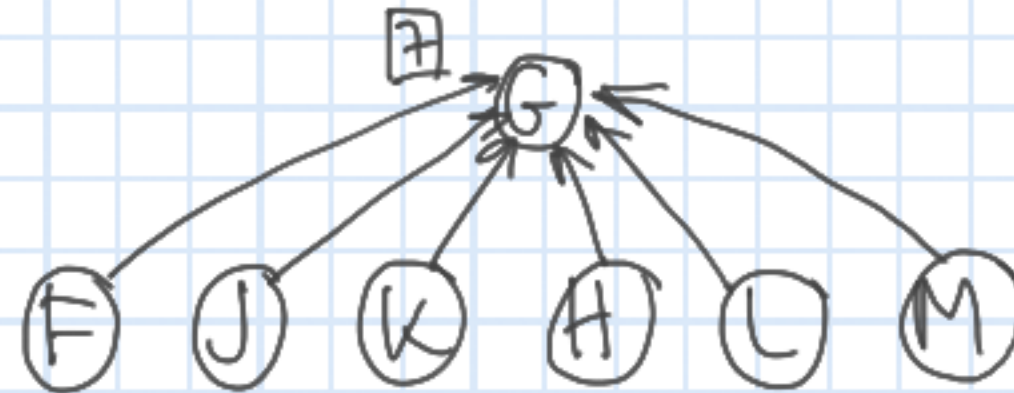
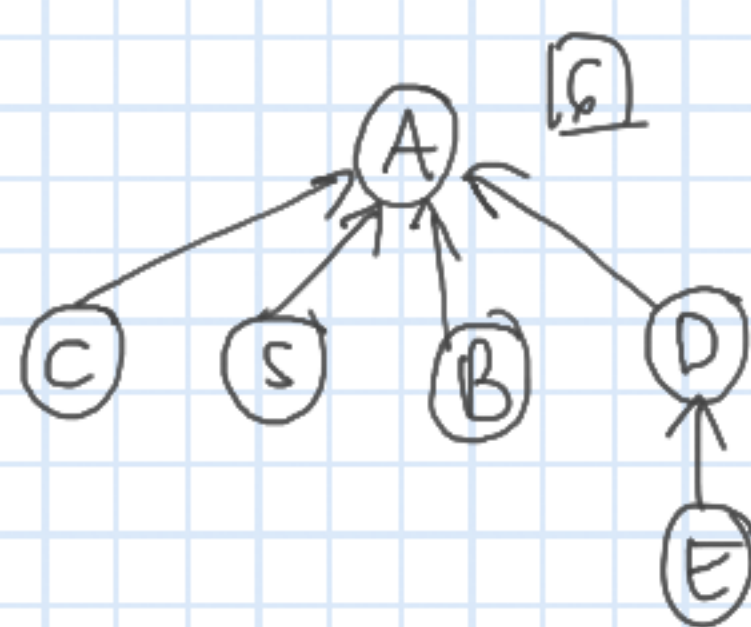
N^1



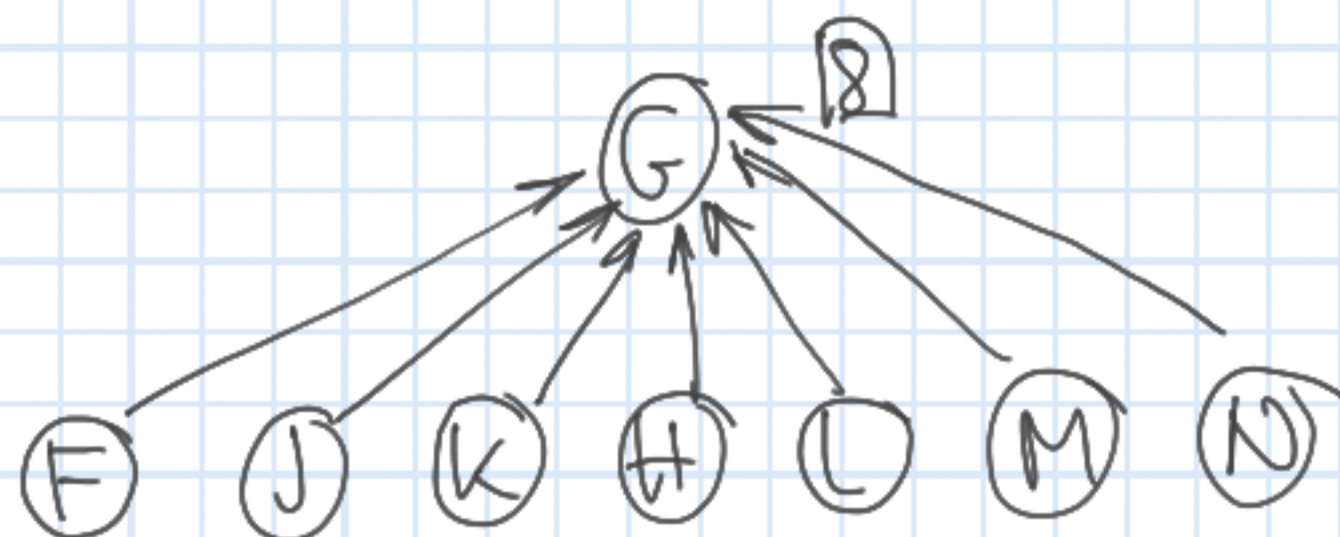
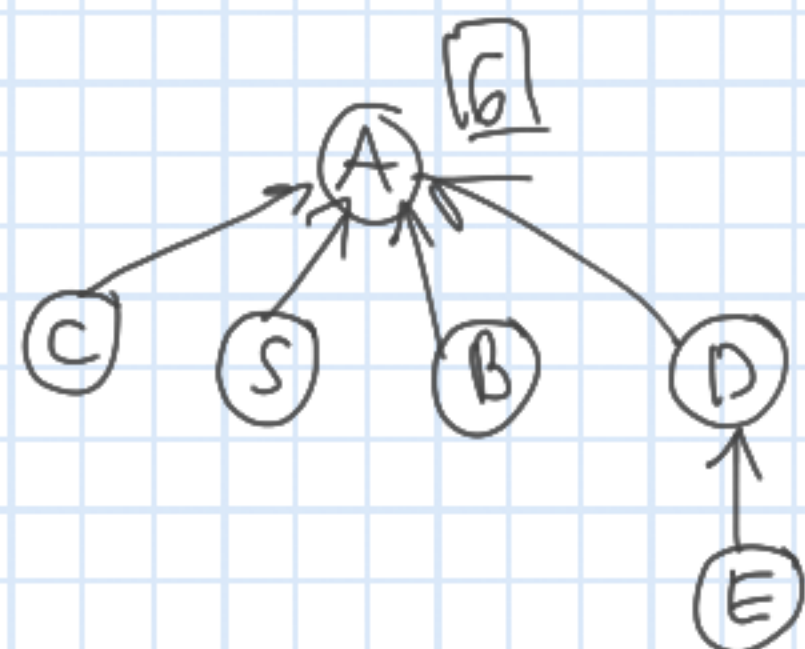
N^1



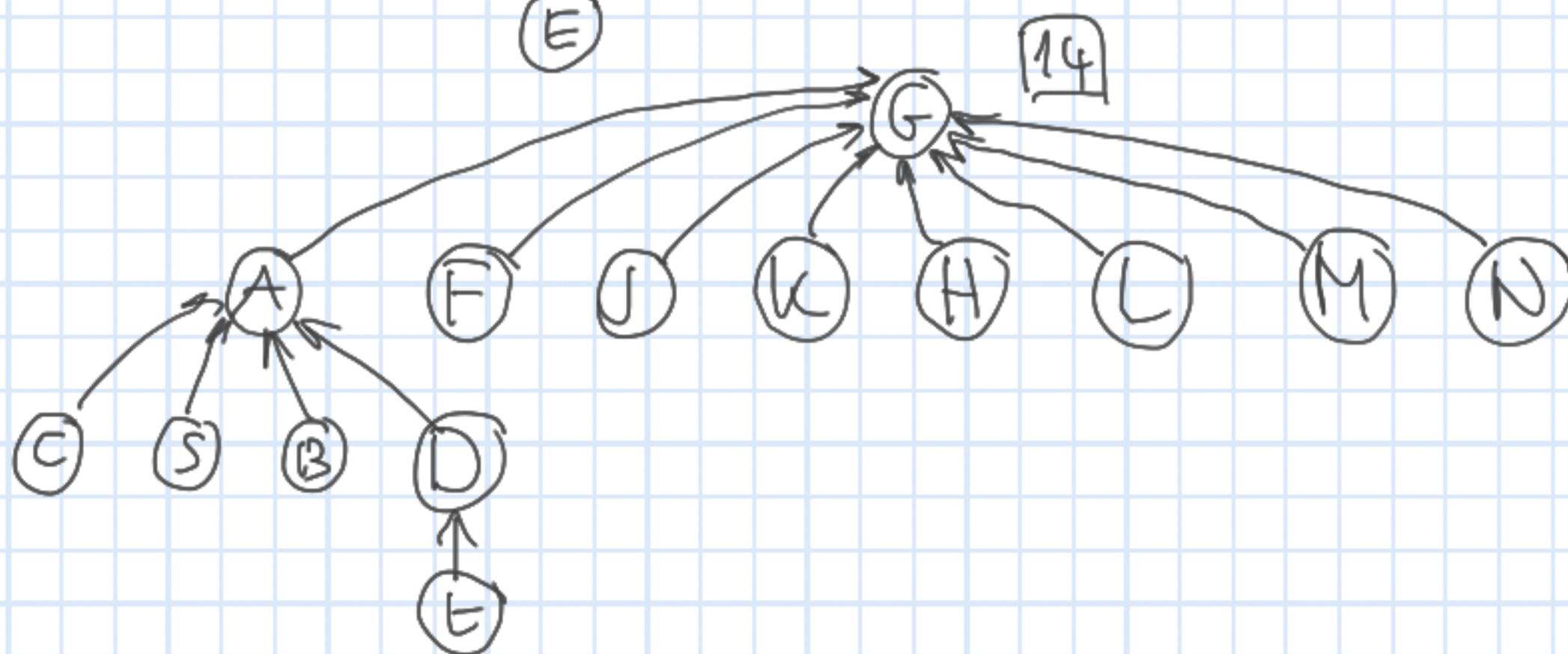
N^1



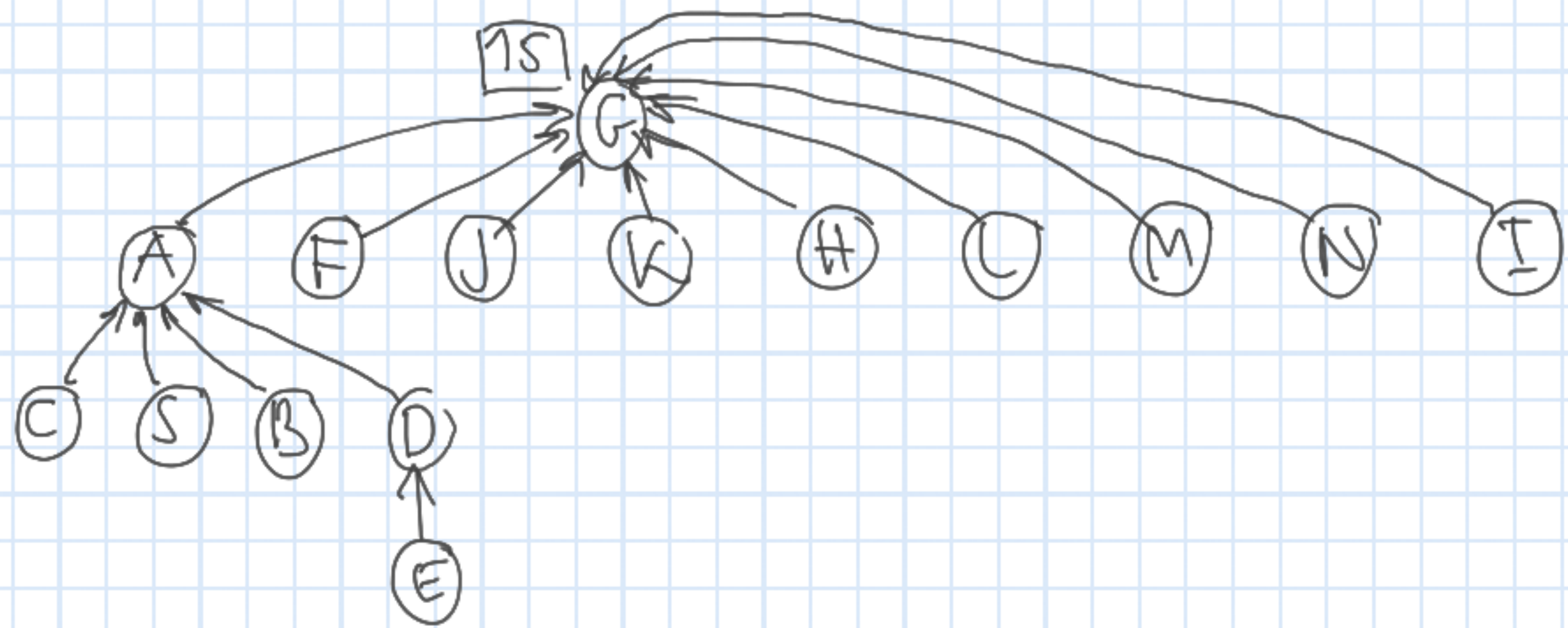
... (N) 1



(I) 1



(I) 1



Problem (Hw)

Prove that if the edge weights are pairwise distinct then the minimum spanning tree is unique.

Shortest paths in weighted graphs

We are given a directed or undirected weighted graph $G = (V, E)$ with weight function $w: E \rightarrow \mathbb{R}$

Suppose, for the sake of simplicity, that G is given by its adjacency matrix

$$C[u, v] = \begin{cases} 0 & \text{if } u = v \\ w(u, v) & \text{if } u \neq v \text{ and } (u, v) \in E \\ \infty & \text{otherwise} \end{cases}$$

Also, usually we take $V = \{1, 2, \dots, n\}$

The length of a path from a vertex u to a vertex v is defined as the sum of the weights of the edges belonging to this path.

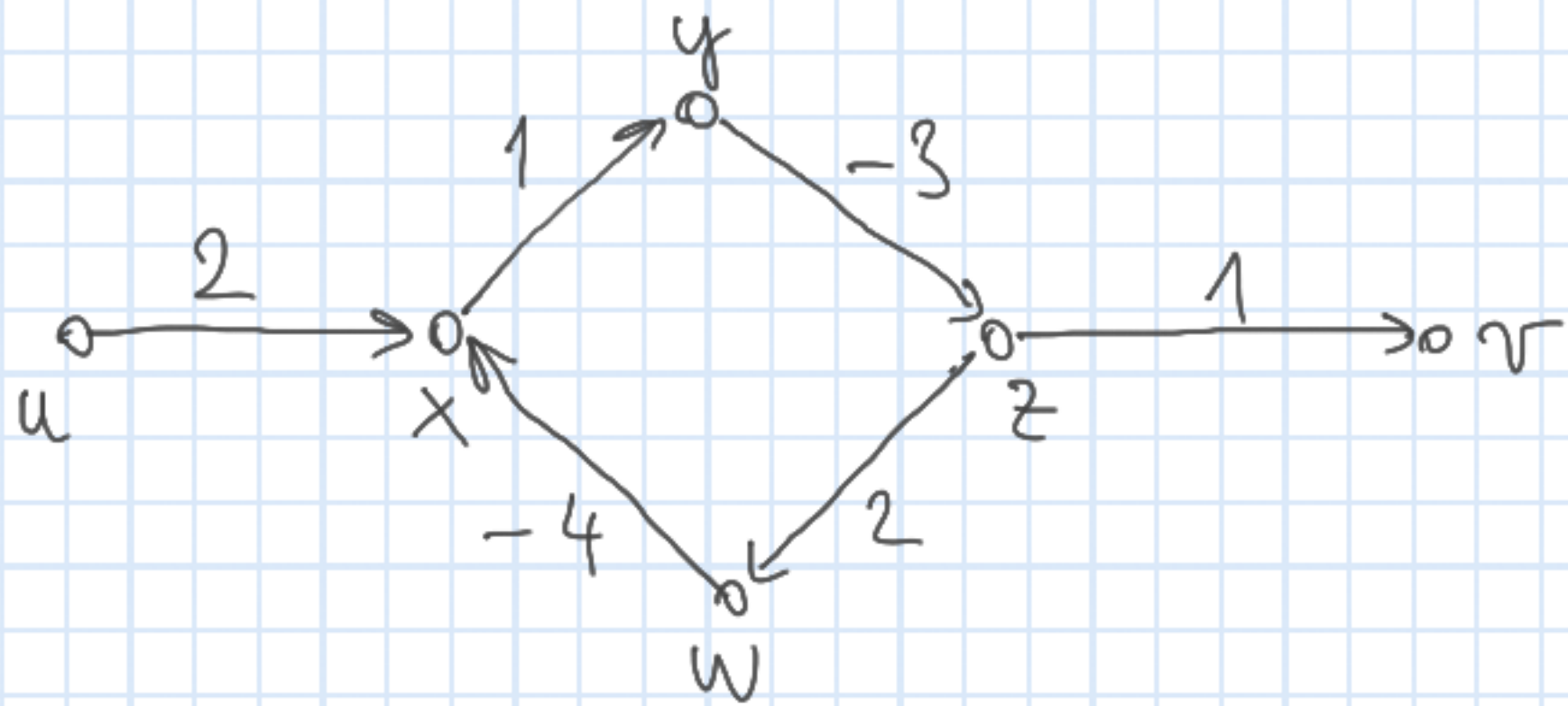
A shortest path from u to v is a path from u to v of smallest length

We can define the distance of u and v as

$$d(u, v) = \begin{cases} 0 & \text{if } u = v \\ \infty & \text{if there is no path from } u \text{ to } v \\ \text{the length of a shortest path from } u & \text{otherwise} \\ & \text{to } v \end{cases}$$

Be careful, $d(u, v) \neq d(v, u)$, i.e., the distance is not symmetric

Also, $d(u, v)$ is not well defined in some cases



length of (u, x, y, z, v) : 1

length of $(u, x, y, z, w, x, y, z, v)$: -3

length of $(u, x, y, z, w, x, y, z, w, x, y, z, v)$: -7

and so on, the lengths are not bounded from below.

From now on we will assume that G doesn't contain negative length cycles, thus the distances are well defined.

BUT

How complex is to decide whether a graph contains a negative cycle? Is there an efficient algorithm to answer this question?

